

Міністерство освіти і науки України
Національний університет «Запорізька політехніка»

Кафедра програмних засобів

ЗВІТ

Дисципліна «Інженерія програмного забезпечення вбудованих систем»

Самостійна робота

Виконав варіант 19

Студент КНТ-122

Онищенко О.А.

Прийняли

Викладач

Пархоменко А.В.

2025

1 ОСНОВИ РОБОТИ З PROCESSING

Мета

У рамках цієї роботи необхідно ознайомитись з базовими поняттями Processing, вивчити основні функції для побудови графічних елементів і роботи з ними.

Теорія

Processing є інтегрованим середовищем розробки (IDE) з відкритим кодом для створення цифрового мистецтва та інтерфейсів користувача. (Джерело: [Processing at Download](#)).

Програма є портативною (її не треба встановлювати). Після завантаження та розпакування розробка здійснюється модифікованою мовою Java, формат файлу .pde. Програму можна запустити або за кнопкою, або скороченням Ctrl+R. Основні функції мови: `setup` (для налаштування вікна), `draw` (головний цикл), `stroke` (для задавання кольору обведення),

Код

1 Фігура

```
void setup() {  
  size(400, 400);  
}  
  
void draw() {  
  background(255);  
  fill(120, 77, 32, 77);  
  noStroke();  
  
  // body  
  ellipse(200, 200, 70, 80);  
  rect(177, 108, 40, 52);  
}
```

```

stroke(120, 77, 32, 77);
strokeWeight(3);

// arms & legs
line(171, 177, 132, 132);
line(231, 183, 132, 132);
line(189, 237, 180, 292);
line(210, 237, 230, 292);

// treasure
stroke(253, 200, 63);
line(120, 112, 120, 147);
line(110, 123, 130, 123);

stroke(120, 77, 32, 77);
fill(255);

// face
ellipse(187, 127, 12, 12);
ellipse(207, 129, 12, 12);
arc(197, 147, 27, 12, 0, PI);

fill(0, 0, 0, 73);
noStroke();

ellipse(187, 127, 5, 5);
ellipse(207, 129, 5, 5);
}

```

2 Кола

```

void setup() {
  size(412, 412);
}

void draw() {
  background(255);

  int opacity = 255;
  fill(255, 89, 94, opacity);
  int border = 0;
  strokeWeight(border);
  stroke(255, 89, 94, 37);
  ellipse(100, 100, 70, 70);

  opacity -= 75;
  fill(138, 201, 38, opacity);
  border += 10;
  strokeWeight(border);
  stroke(138, 201, 38, 37);
  ellipse(200, 200, 70, 70);

  opacity -= 75;
  fill(25, 130, 196, opacity);
  border += 10;
}

```

```
strokeWeight(border);  
stroke(25, 130, 196, 37);  
ellipse(300, 300, 70, 70);  
}
```

Результати

1 Фігура

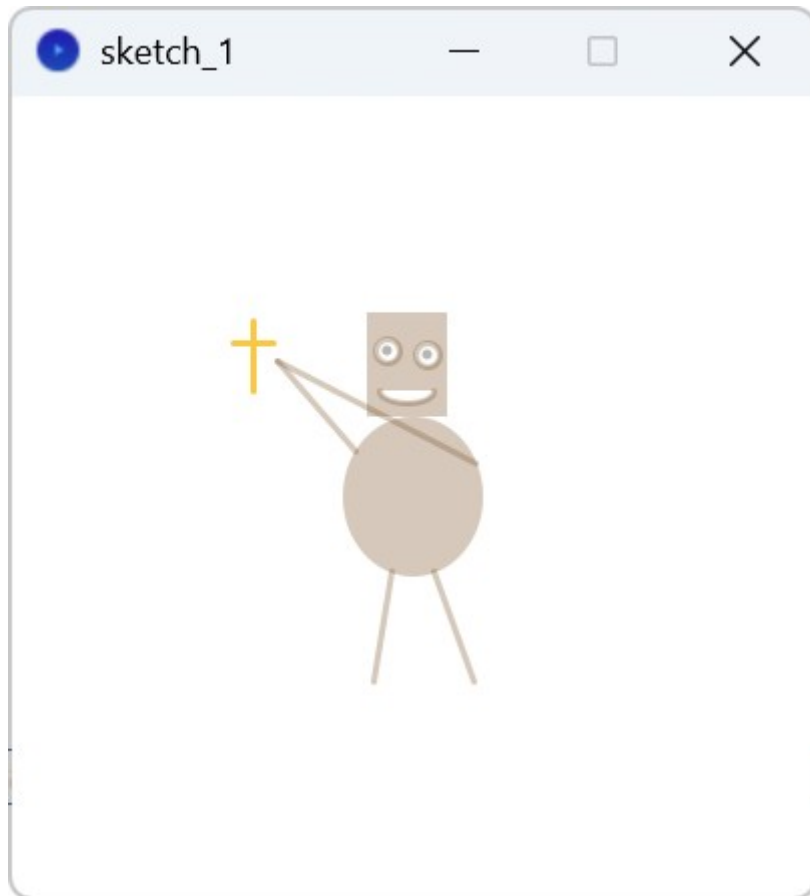


Рисунок 1.1 – Вигляд програми

2 Кола

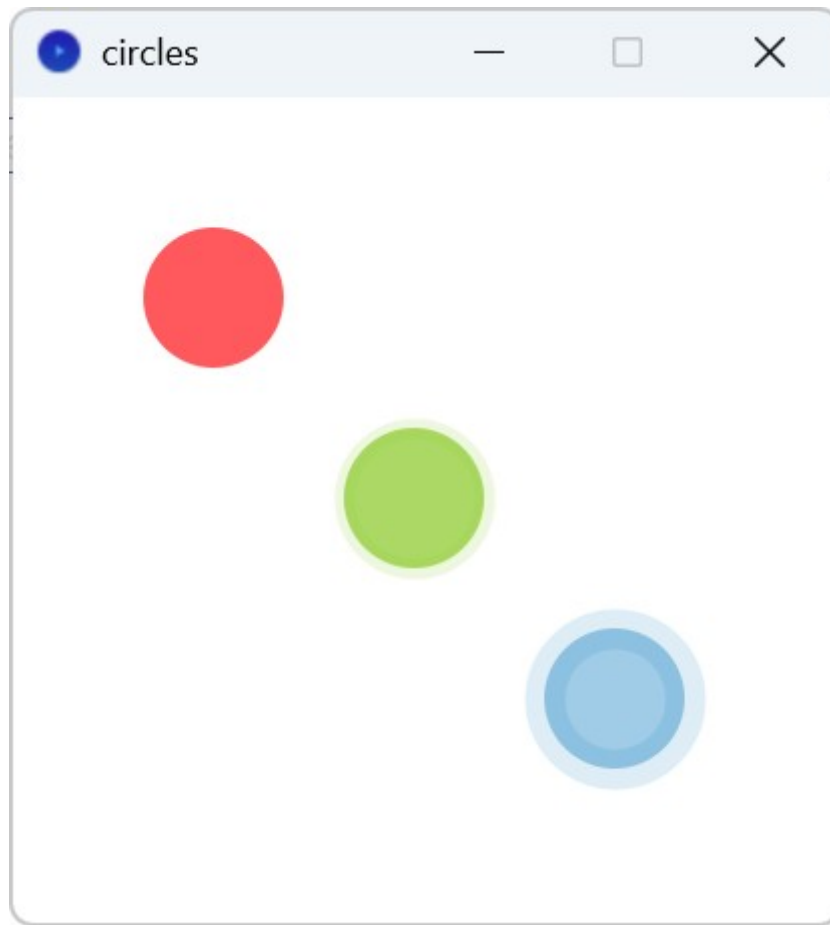


Рисунок 2.1 – Вигляд результатів

Висновки

В результаті виконання першої частини самостійної роботи благодаттю Господа нашого Ісуса Христа було ознайомлено з базовими поняттями Processing, та вивчено основні функції для побудови графічних елементів і роботи з ними.

Питання

Які параметри робочого вікна встановлені за замовчанням?

За замовчуванням Processing встановлює такі параметри типових методів:

- Розмір вікна: 100 * 100;
- Фон: світло-сірий;
- Заливка: біла;
- Контур: чорний.

Джерело: [size\(\) / Reference / Processing.org](#).

Якими будуть значення функції заповнення для чорного, білого та середнього сірого кольору?

Значення функції для заповнення мають бути такі:

- Сірий: 127;
- Білий: 255;
- Чорний: 0.

Який з нижче наведених еліпсів буде найпрозорішим?

```
fill(255, 0, 0, 90);  
ellipse(100, 100, 150, 75);  
  
fill(0, 0, 0, 100);  
ellipse(50, 50, 120, 60);  
  
fill(0, 100, 0, 200);  
ellipse(50, 80, 60, 60);
```

Перший еліпс буде найпрозоріший, бо останнє значення fill найменше.

Вкажіть довжину у сантиметрах лінії:

```
line(0, 50, 50, 50);
```

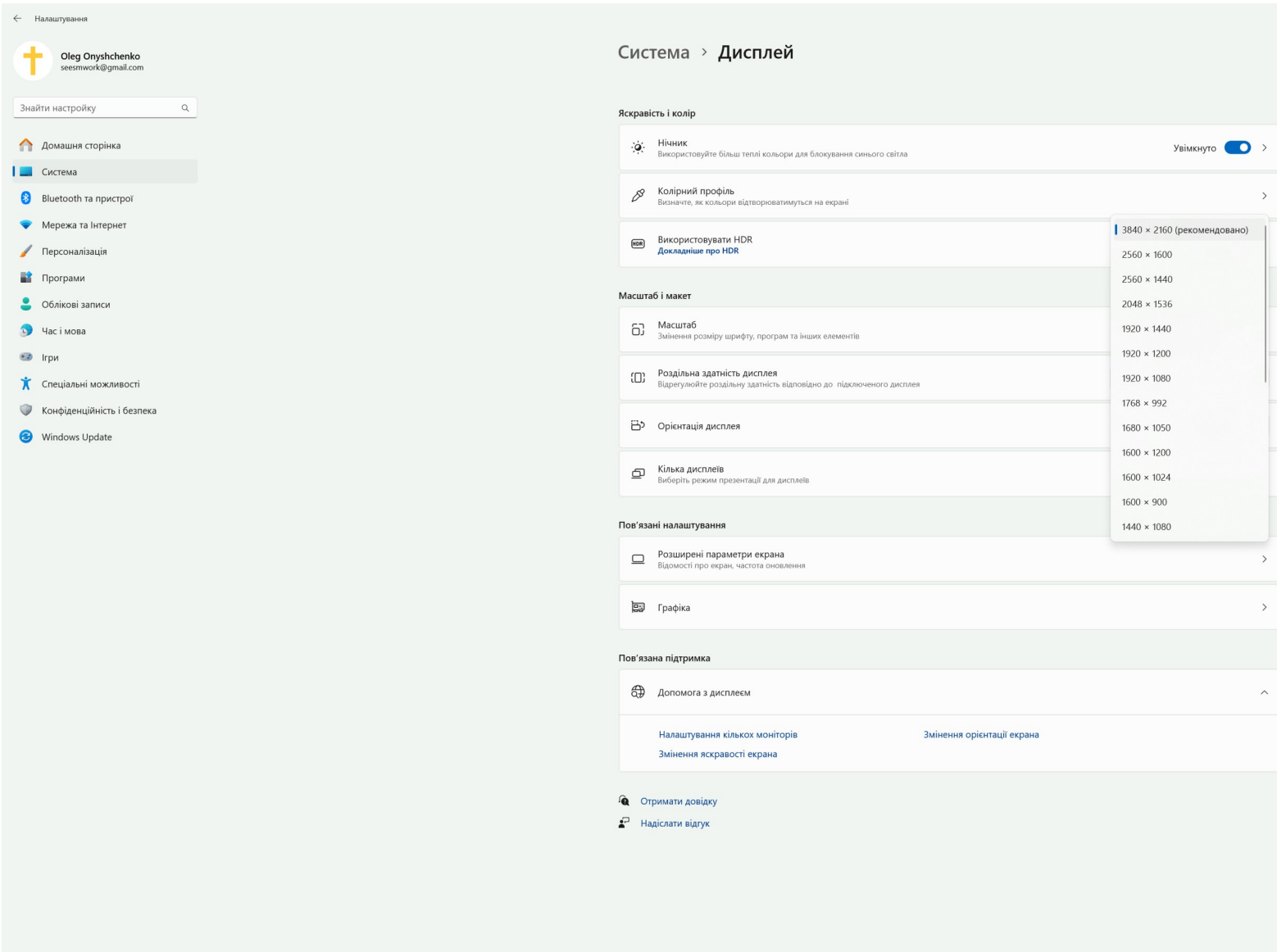


Рисунок 0.1 – Роздільна здатність монітора в налаштуваннях

Розмір монітора в пікселях: 3840 * 2160

Розмір монітора: 28"

Розрахунки:

$$3840^2 + 2160^2$$

$$3840^2 = 14745600$$

$$2160^2 = 4665600$$

$$4665600 + 14745600 = 19411200$$

$$\sqrt{19411200} \approx 4405$$

$$4405 / 28 \approx 157.3$$

$$157.3 \cdot 2.54 \approx 399.5$$

$$399.5 / 50 / 10 \approx 0.8$$

Відповідь: довжина лінії у 50 пікселів шириною приблизно дорівнює восьми міліметрам або нуль цілим восьми десятим сантиметра.

2 РОБОТА ЗІ ЗМІННИМИ У PROCESSING

Мета

Ознайомитись з поняттям змінної, які змінні існують у середовищі Processing за замовчанням. Навчитися створювати власні змінні та застосовувати їх при написанні програм.

Теорія

Змінна є йменованим місцем у пам'яті. Місце займається програмою і може змінювати своє значення під час виконання.

Змінні можуть мати різні області видимості. Локальна змінна доступна лише у своєму блоці коду, а глобальна у всіх, що нижче.

Приклад локальної змінної:

```
static void functionCalled() {  
    int namedVariable = 7;  
    System.out.println(namedVariable);  
}  
> 7
```

Приклад глобальної змінної

```
char anotherNamedPlace = 'T';  
  
static void showVariable() {  
    System.out.println(anotherNamedPlace);  
}  
> T
```

Джерело: [Variable in Programming - GeeksforGeeks](#)

Результати

1 Коло зліва направо

Розробити програму, яка відображає коло, яке переміщується з лівої сторони робочого вікна Processing до правої, і коли досягає крайнього правого положення знову починає рухатись зліва на право.

[Відео.](#)

При розробці було додано параметр швидкості. Він визначає наскільки буде збільшуватись позиція кульку за один цикл. Але не було враховано те, що позиція може перевищувати ширину екрану, тому вона зупинялась на правому боці.

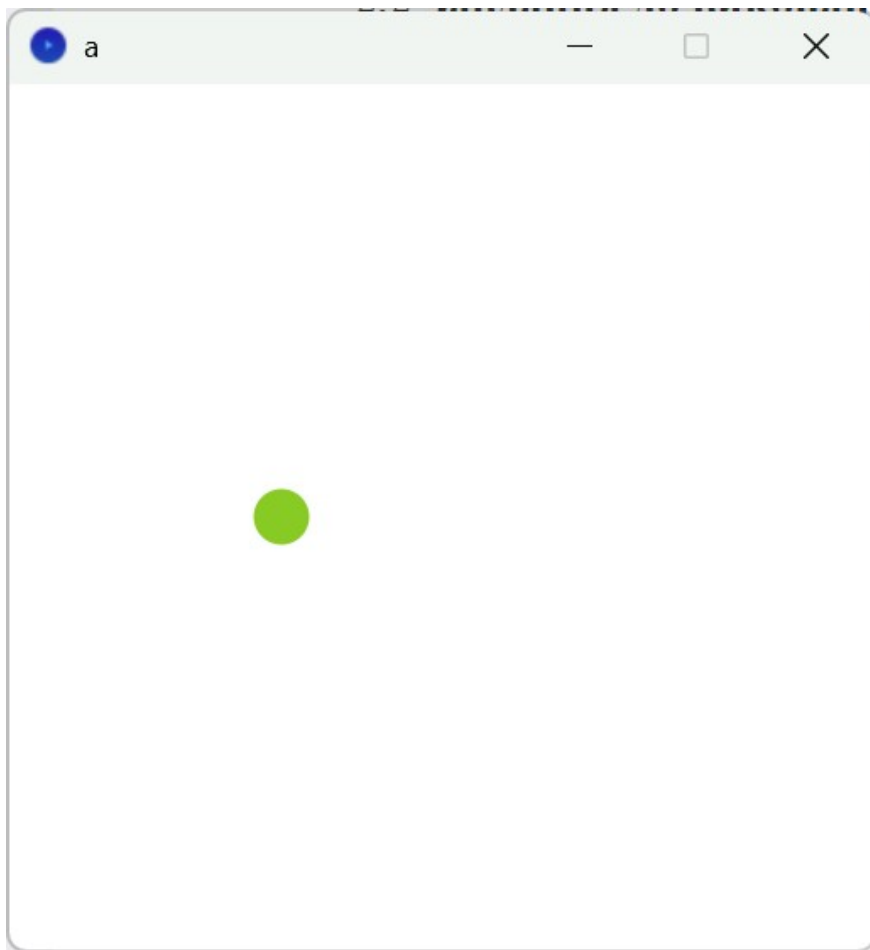


Рисунок 1.1 – Програма в роботі

Код програми

```
int circleHorizontal=0;
int speed=7;
boolean goingRight=true;
int circleHeight=33;

void setup() {
  size(512, 512);
}

void draw() {
  background(255);
  noStroke();
  fill(137, 203, 37);

  ellipse(circleHorizontal, height/2, circleHeight, circleHeight);
  if (goingRight && circleHorizontal-circleHeight/2<width-circleHeight)
  {
    circleHorizontal+=speed;
  } else if (goingRight && circleHorizontal>=width-circleHeight*2) {
    goingRight=false;
    circleHorizontal-=speed;
    println(circleHorizontal);
    println(width-circleHeight*2);
  } else if (!goingRight && circleHorizontal-circleHeight/2>0) {
    circleHorizontal-=speed;
  } else if (!goingRight && circleHorizontal-circleHeight/2<=0) {
    circleHorizontal+=speed;
    goingRight=true;
  }
}
```

2 Різні прямокутники

Розробити програму, яка створює робоче вікно світло-блакитного кольору з двома прямокутниками по центру, один з яких поступово змінює колір з білого до чорного, а інший з чорного на білий і навпаки.

Відео.

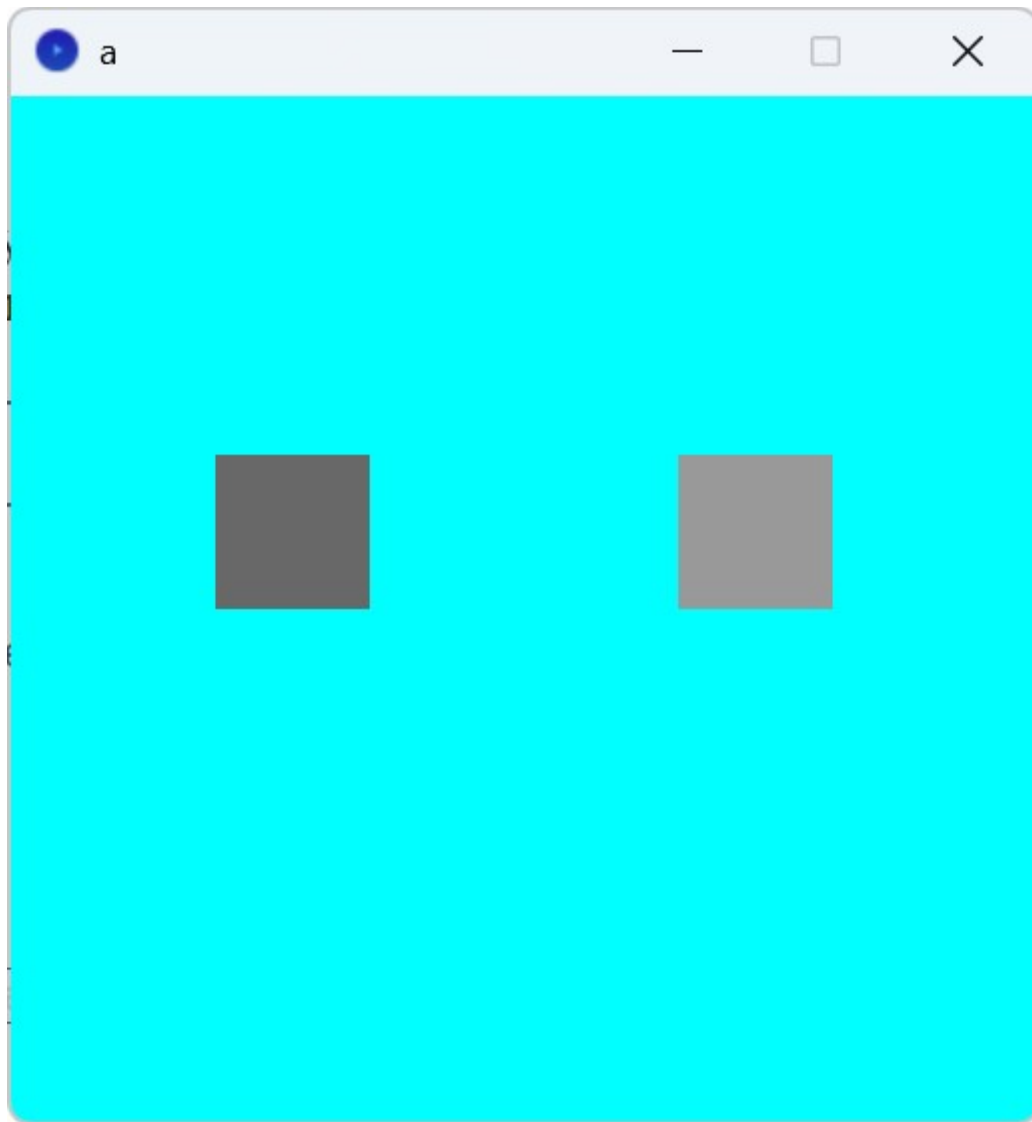


Рисунок 2.1 – Програма в роботі

Код програм

```
int rectangleWidth=77;
int rectangleHeight=77;
int firstColor=255;
boolean firstGoingUp=false;
int secondColor=0;
boolean secondGoingDown=false;
int speed=1;

void setup() {
    size(512, 512);
    background(0,255,255);
}

void draw() {
```

```

noStroke();

fill(firstColor);
rect(width/2-rectangleWidth*2, height/2-rectangleHeight,
rectangleWidth, rectangleHeight);
if (!firstGoingUp && firstColor>=0) {
    firstColor-=speed;
} else if (!firstGoingUp && firstColor<=0) {
    firstColor+=speed;
    firstGoingUp=true;
} else if (firstGoingUp && firstColor<=255) {
    firstColor+=speed;
} else if (firstGoingUp && firstColor>=255) {
    firstColor-=speed;
    firstGoingUp=false;
}

fill(secondColor);
rect(width/2+rectangleWidth, height/2-rectangleHeight,
rectangleWidth, rectangleHeight);
if (!secondGoingDown && secondColor>=0) {
    secondColor-=speed;
} else if (!secondGoingDown && secondColor<=0) {
    secondColor+=speed;
    secondGoingDown=true;
} else if (secondGoingDown && secondColor<=255) {
    secondColor+=speed;
} else if (secondGoingDown && secondColor>=255) {
    secondColor-=speed;
    secondGoingDown=false;
}
}

```

3 Лінія за курсором

Розробити програму, яка буде малювати нескінченну лінію за курсором мишки.

[Відео.](#)

При написанні до циклу було додано зміну кольору фону, через це лінія зникала одразу.

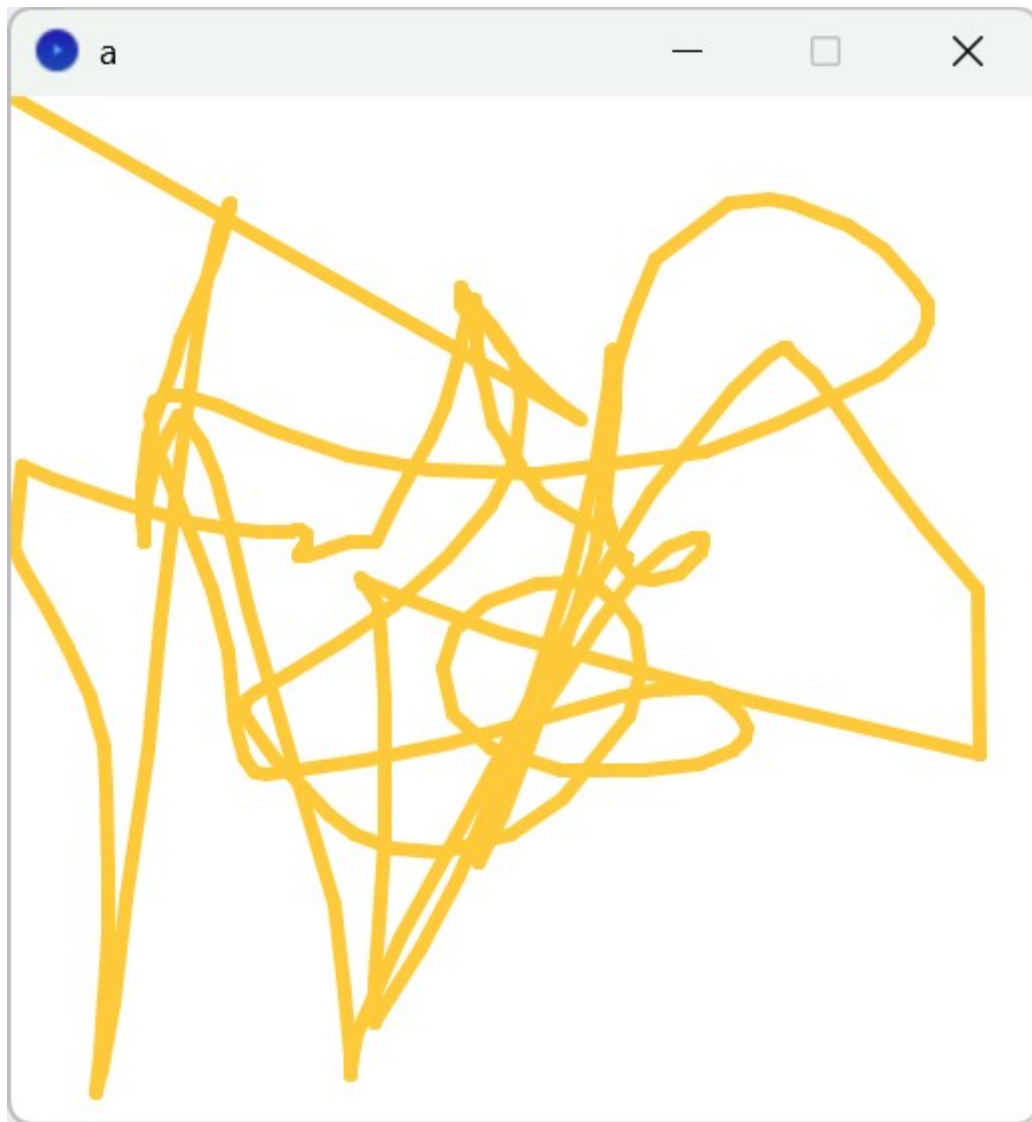


Рисунок 3.1 – Програма в роботі

Код програм

```
void setup() {  
  size(512, 512);  
  background(255);  
}  
  
void draw() {  
  stroke(255, 202, 58);  
  strokeWeight(7);  
  
  line(pmouseX, pmouseY, mouseX, mouseY);  
}
```

4 Кутове коло

Розробити програму, яка відображає коло, яке циклічно переміщується з крайньої лівої сторони робочого вікна Processing до крайньої правої сторони і в зворотному напрямку.

[Відео.](#)

При написанні було переплутано порівняння ширини з висотою, тому м'ячик зупинявся на нижньому краю. Також два рази було порівнювано зі значеннями ширини і висоти екрану, а не з нулем, тому м'ячик теж застрявав на краю екрану.

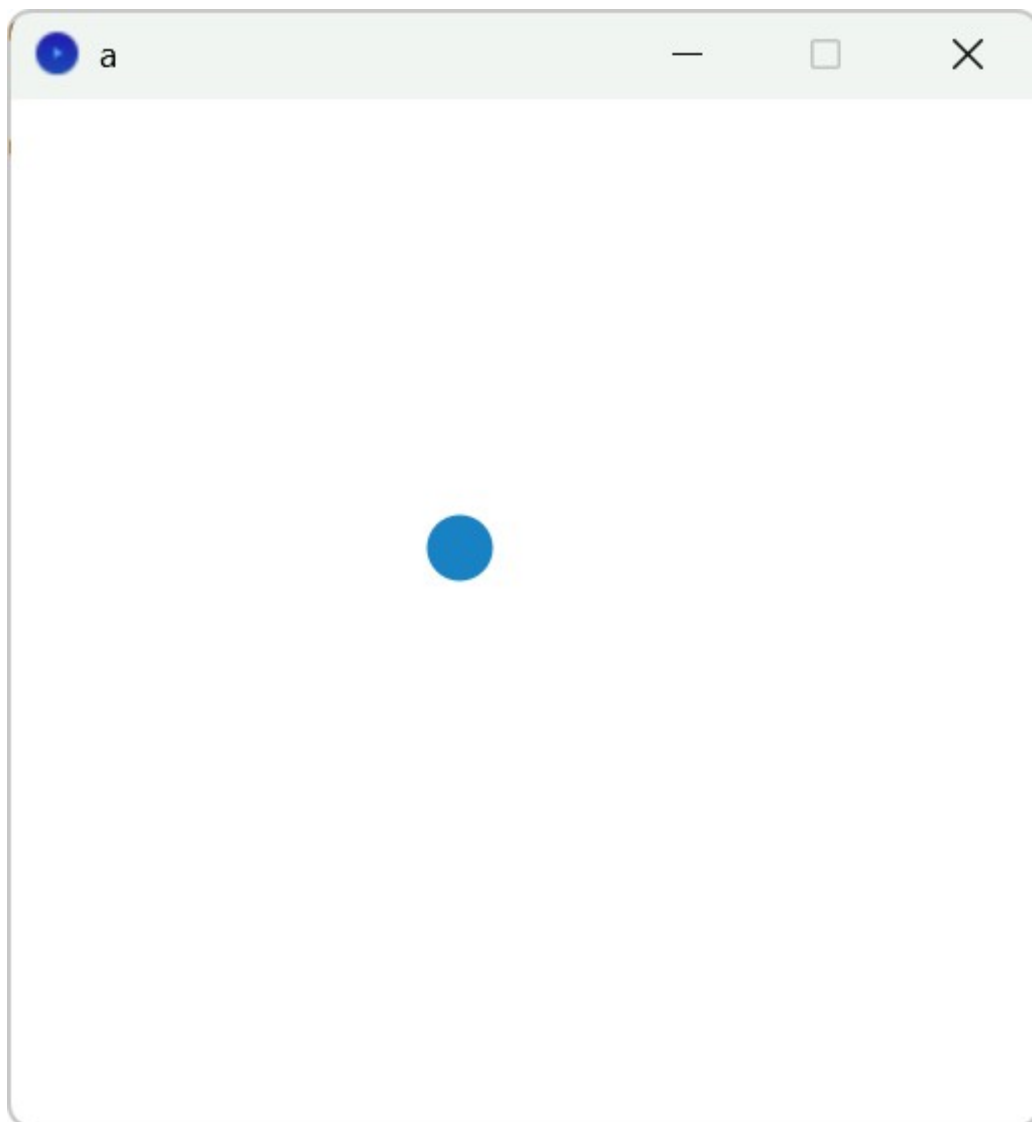


Рисунок 4.1 – Програма в роботі

Код програм

```
int horizontal=0;
int vertical=0;
int speed=7;
boolean goingUp=false;

void setup() {
  size(512, 512);
}

void draw() {
  background(255);
  noStroke();
  fill(25, 130, 196);

  ellipse(horizontal, vertical, 33, 33);
  if (!goingUp && horizontal<width && vertical<height) {
    horizontal+=speed;
    vertical+=speed;
  } else if (!goingUp && horizontal>=width && vertical>=height) {
    goingUp=true;
    horizontal-=speed;
    vertical-=speed;
  } else if (goingUp && horizontal>0 && vertical>0) {
    horizontal-=speed;
    vertical-=speed;
  } else if (goingUp && horizontal<=0 && vertical<=0) {
    goingUp=false;
    horizontal+=speed;
    vertical+=speed;
  }
}
```

Висновки

Благодаттю Господа нашого Ісуса Христа було ознайомлено з поняттям змінної, які змінні існують у середовищі Processing за замовчанням; навчено створювати власні змінні та застосовувати їх при написанні програм.

Питання

1 Змінна, її типи

Що таке змінна? Змінна є йменованим місцем у пам'яті, яке використовується програмою для повторного використання значень.

Які бувають змінні? Змінні бувають глобальними і локальними. Змінні також можуть бути в ролі констант, зберігати незмінні значення.

2 Ініціалізація змінної, які символи не мають бути в назві

Як ініціалізувати змінну? Для ініціалізації потрібно через знак дорівнює вказати її значення, яке відповідає оголошеному типу. (Джерело: [Variables / Examples / Processing.org](#))

Які символи не повинні міститись у назві змінної? На початку не має бути цифр і символу #. Назва змінної не має бути ключовим словом мови програмування.

3 Різниця між setup та draw

Опишіть різницю між роботою функцій void setup() та void draw()? Функция setup виконується один раз при запуску програми, а функція draw є «нескінченим» циклом, який триває весь час виконання програми. (Джерело: [Setup and Draw / Examples / Processing.org](#))

4 Значення змінних mouseX

Як ви зрозуміли роботу змінних `rmouseX` та `rmouseY`? Значення `rmouseX` та `rmouseY` є значеннями змінних `mouseX` та `mouseY` але з попереднього циклу.

3 ВЗАЄМОДІЯ PROCESSING З КОРИСТУВАЧЕМ

Мета

Навчитися програмного реалізовувати взаємодію користувача з Processing за допомогою таких подій як: клік миші та натиснення клавіші на клавіатурі.

Завдання

1. Створите робоче вікно розмірами 640 на 480 пікселів чорного кольору. Розділити цю область двома лініями білого кольору на чотири частини. Коли курсор миші переміщується над якоюсь частиною то вона змінює колір з чорного на будь-який інший.

2. Створите робоче вікно розмірами 480 на 120 пікселів та біле коло розмірами 50 на 50, яке рухається за курсором миші. Коли клавіша миші натиснута, колір кола змінюється на чорний.

3. Модифікуйте програму з колом, яке рухається. Зробіть так, що коли клавіша миші натиснута в перший раз, коло починає рухатись, коли клавіше миші натиснута вдруге, коло зупиняється.

Теорія

В Processing можна отримати доступ до подій натисканні клавіш через функції `mousePressed` і `buttonPressed`.

`mousePressed` показує чи натиснута кнопка миші. `buttonPressed` показує чи натиснута кнопка клавіатури.

`mouseButton` повертає яка саме кнопка миші була натиснута.

Виконання

1 Змінні квадрати

```
void setup() {
    size(640, 480);
    background(255);
}

void resetBackground() {
    fill(255);
    // up left
    rect(0,0,width/2,height/2);
    // up right
    rect(width/2,0,width,height/2);
    // down left
    rect(0,height/2,width/2,height/2);
    // down right
    rect(width/2,height/2,width,height);

    fill(0);
    line(width/2, 0, width/2, height);
    line(0, height/2, width, height/2);
}

void draw() {
    println(mouseX,mouseY);
    if (mouseX<width/2 && mouseY<height/2) {
        resetBackground();
        fill(255,255,0);
        rect(0, 0, width/2, height/2);
    } else if (mouseX>=width/2 && mouseY<height/2) {
        resetBackground();
        fill(255,255,0);
        rect(width/2, 0, width, height/2);
    } else if (mouseX<width/2 && mouseY>=height/2) {
        resetBackground();
        fill(255,255,0);
        rect(0, height/2, width/2, height);
    } else if (mouseX>=width/2 && mouseY>=height/2) {
        resetBackground();
        fill(255,255,0);
        rect(width/2, height/2, width, height);
    }
}
```



Рисунок 1.1. - Програма в роботі

Відео: https://youtu.be/BQB_0kk6l88.

2 Змінне коло

```
void setup() {  
  size(480, 120);  
}  
  
void draw() {  
  background(94, 141, 223);  
  println(mouseX, mouseY);  
  noStroke();  
  
  if (!mousePressed) fill(255);  
  else fill(0);  
  ellipse(mouseX, mouseY, 50, 50);  
}
```

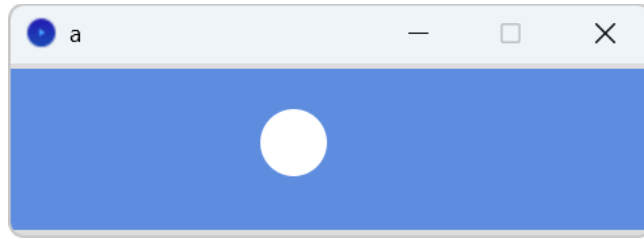


Рисунок 2.1 - Програма в роботі

Відео: https://youtu.be/Oy_x7y6-hxE.

3 Коло із зупинками

```
boolean stopped=false;
int circleSize=50;
int circleHorizontal=0;
int speed=3;
boolean goingRight=true;

void setup() {
  size(480, 320);
}

void draw() {
  background(141, 255, 64);
  noStroke();
  fill(255);

  if (!stopped) {
    if (goingRight && circleHorizontal<width-circleSize/2) {
      circleHorizontal+=speed;
    } else if (goingRight && circleHorizontal>=width-circleSize/2) {
      goingRight=false;
      circleHorizontal-=speed;
    } else if (!goingRight && circleHorizontal>0+circleSize/2) {
      circleHorizontal-=speed;
    } else if (!goingRight && circleHorizontal<=0+circleSize/2) {
      goingRight=true;
      circleHorizontal+=speed;
    }
  }
  ellipse(circleHorizontal, height/2, circleSize, circleSize);
}

void mousePressed() {
  stopped=!stopped;
}
```

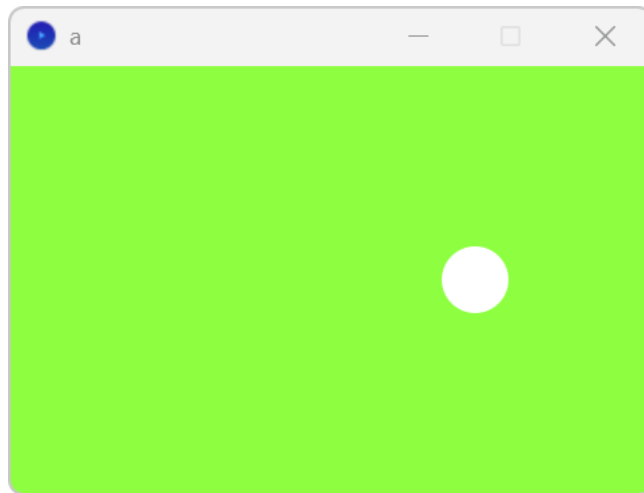


Рисунок 3.1 - Програма в роботі

Відео: <https://youtu.be/xVJ9yTPnIso>.

Висновки

По виконанню, благодаттю Господа нашого Ісуса Христа, роботу було навчено програмно реалізовувати взаємодію користувача з Processing за допомогою подій кліка миші та натиснення клавіш на клавіатурі.

Питання

mousePressed як функція та як змінна, принципи роботи?

Як функція вона спрацьовує один раз коли було натиснуто кнопку миші. Як змінна вона змінює значення на true весь час коли кнопка натиснута.

keyPressed як функція та як змінна, принципи роботи?

Як функція спрацьовує один раз при натисканні, як змінна змінює значення на true поки будь-яка клавіша затиснута.

Обидві можуть працювати лише в методі draw().

В чому помилка у наступній програмі? Чому не відображається коло на екрані?

```
float circleX=0;

void setup() {
  size(640, 360);
}

void draw() {
  background(50);
  fill(255);
  ellipse(circleX, 180, 24, 24);
  circleX=circleX+random(-2, 0);
}
```

Помилка в тому, що згенероване «випадкове» значення є майже завжди негативним, тому горизонтальна позиція зменшується і коло рухається в напрямку вліво від екрану.

4 ПОСЛІДОВНИЙ ІНТЕРФЕЙС ВВОДУ/ВИВОДУ

Мета

На простих прикладах освоїти принципи роботи з монітором послідовного порту середовищ розробки Processing та Arduino.

Завдання

1. Ознайомитися з теоретичними відомостями.
2. Допрацюйте у Processing програму з колом, яке рухається, з самостійної роботи №2, щоб координати центра кола виводилися до монітору послідовного порту Processing.
3. В Arduino розробити програму, яка зчитує значення, яке вводяться користувачем у моніторі послідовного порту та відповідно до нього встановлює положення серво підключеного до плати Arduino.
4. Оформити звіт до самостійної роботи.

Теорія

У Processing виводити дані можна методами `print` або `println`. Метод `print` просто виводить рядок, а `println` додає новий рядок в кінці.

У Tinkercad для запуску монітору потрібно використати `Serial.begin()`. Буфер можна перевірити на наявність символів методом `Serial.available()`. Прочитати рядок можна методом `Serial.readString()`. Прочитати число можна методом `Serial.parseInt()`.

Виконання

1 Координати кола

```
int horizontal=0;
int vertical=0;
int speed=7;
boolean goingUp=false;

void setup() {
  size(512, 512);
}

void draw() {
  background(255);
  noStroke();
  fill(25, 130, 196);

  ellipse(horizontal, vertical, 33, 33);
  println(horizontal, vertical);
  if (!goingUp && horizontal<width && vertical<height) {
    horizontal+=speed;
    vertical+=speed;
  } else if (!goingUp && horizontal>=width && vertical>=height) {
    goingUp=true;
    horizontal-=speed;
    vertical-=speed;
  } else if (goingUp && horizontal>0 && vertical>0) {
    horizontal-=speed;
    vertical-=speed;
  } else if (goingUp && horizontal<=0 && vertical<=0) {
    goingUp=false;
    horizontal+=speed;
    vertical+=speed;
  }
}
```

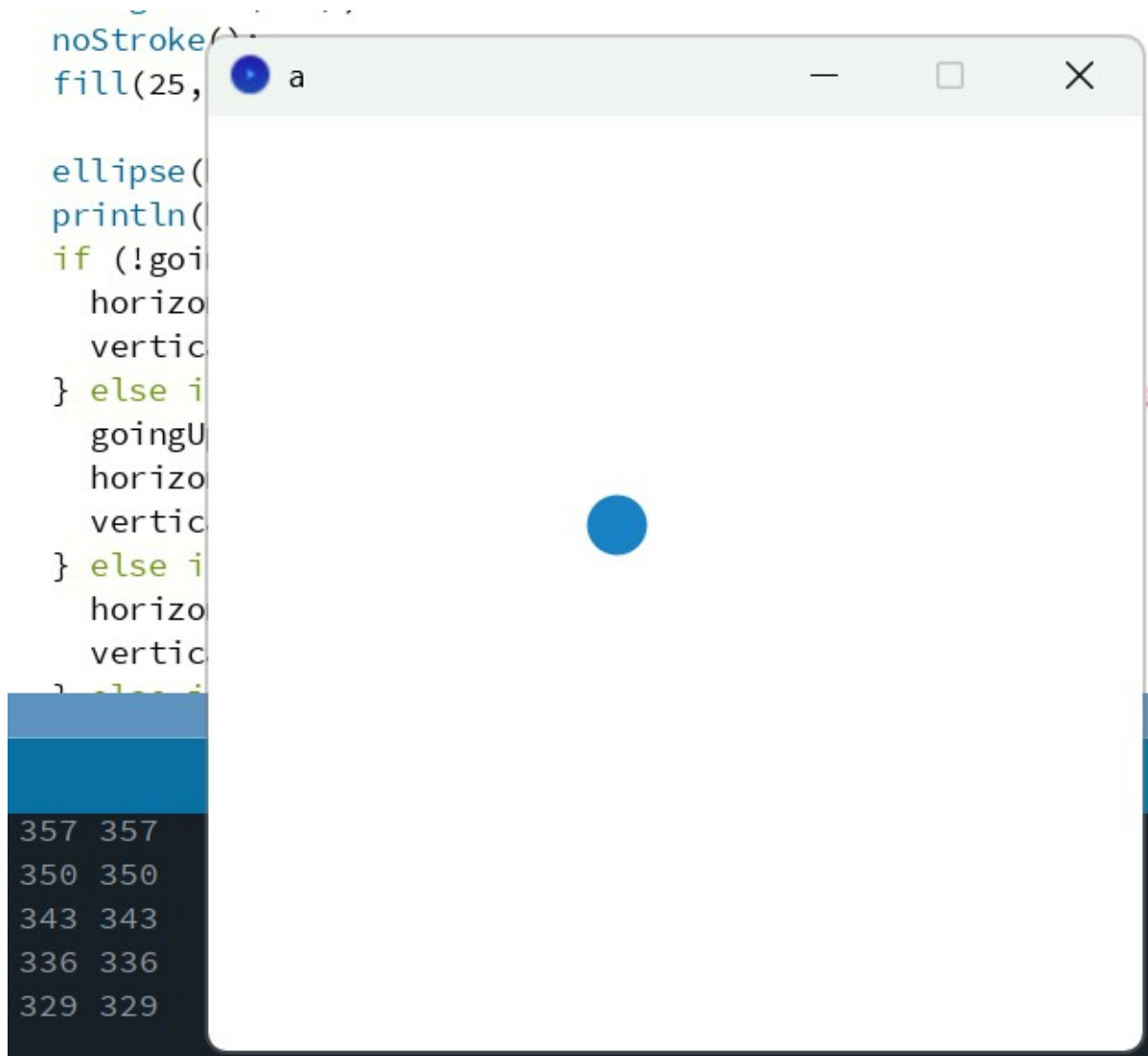


Рисунок 1.1 — Програма в роботі

Відео: <https://youtu.be/O5k18ISFNYM>.

2 Положення серво

```
#include <Servo.h>
Servo servo;

const int servoPin = 12;

void setup()
{
  Serial.begin(9600);
  servo.attach(servoPin);
}
```

```

void loop()
{
  Serial.print("Enter angle. ");
  while (Serial.available()==0) {}
  long angle=Serial.parseInt(SKIP_ALL);
  Serial.println(angle);
  servo.write(angle);
  delay(100);
}

```

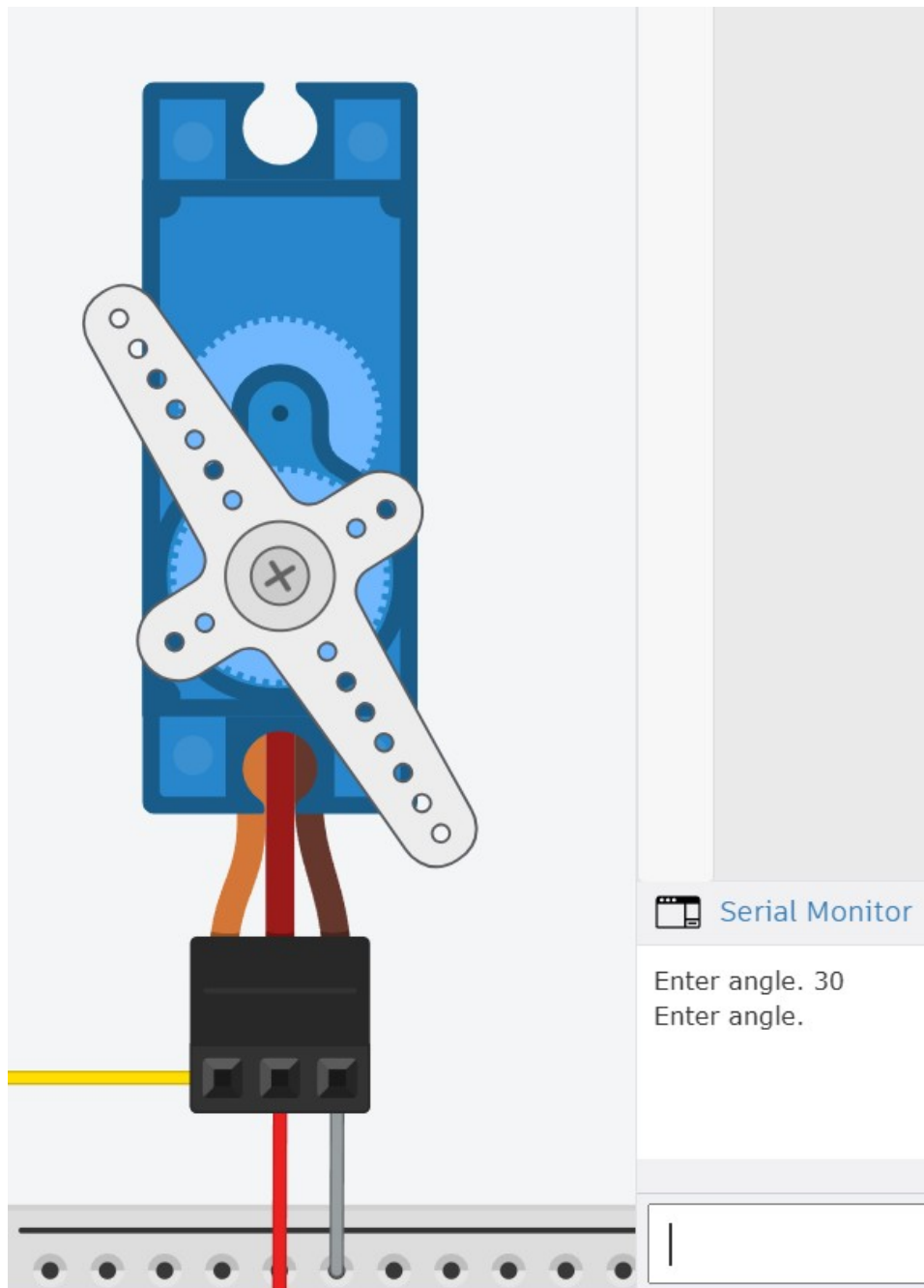


Рисунок 2.1 — Програма в роботі

Відео: https://youtu.be/dMa5j_ZaRCQ.

Висновки

По виконанню, благодаттю Господа нашого Ісуса Христа, роботу на простих прикладах було засвоєно принципи роботи з монітором послідовного порту середовищ розробки Processing та Arduino.

Питання

Назвіть основні команди для роботи з Serial Monitor?

Такі команди є розповсюдженими:

- begin(): починає роботу монітору;
- print(): виводить рядок без нового рядку;
- println(): виводить рядок з новим рядком;
- readString(): читає рядок типу String;
- parseInt(): читає ціле число.

Яка відмінність між print та println?

print не додає нового рядку в кінці: якщо вивести щось знову, воно буде на одному рядку. println додає новий рядок.

З якою швидкістю передаються данні за замовчанням в Arduino?

Значення за замовчуванням — 9600.

Які стандартні швидкості передачі даних визнаєте?

Стандартними є:

- 9600, використовується часто і передає 9 600 бітів за секунду;
- 19200, передає 19 200 біт за секунду.

Джерело: [Detailed explanation serial port baud rate classification and application – IOT Module Shop Manufacturer Factory](#).

5 ВЗАЄМОДІЯ PROCESSING ТА ARDUINO

Мета

Навчитися створювати у Processing IDE простий графічний інтерфейс який за допомогою послідовного порту «спілкується» з Arduino.

Завдання

1. Підключіть до Arduino RGB світлодіод. У Processing розробіть вікно з трьома квадратами різного кольору, щоб при натисканні на квадрат заданого кольору, параметри цього кольору передавалися до Arduino та світлодіод загорявся відповідним кольором.

2. Розробіть інтерфейс у Processing до завдання 4.2.3 самостійної роботи №4.

Виконання

1 Колір світлодіоду

Код Processing

Реалізацію взято з: [Button / Examples / Processing.org](#).

```
int padding=10;
boolean redOver=false;
boolean greenOver=false;
boolean blueOver=false;
color redColor = color(255, 0, 0);
color greenColor = color(0, 255, 0);
color blueColor = color(0, 0, 255);
color redHighlightColor = color(255, 100, 100);
color greenHighlightColor = color(100, 255, 100);
color blueHighlightColor = color(100, 100, 255);

void setup() {
  size(300*3, 100*3);
}

void draw() {
```

```

    update(mouseX,mouseY);
    background(255);
    noStroke();
    int size=width/3;

    if (redOver) fill(redHighlightColor);
    else fill(redColor);
    rect(padding, padding, width/3-padding*2, height-padding*2);

    if (greenOver) fill(greenHighlightColor);
    else fill(greenColor);
    rect(padding+size, padding, width/3-padding*2, height-padding*2);

    if (blueOver) fill(blueHighlightColor);
    else fill(blueColor);
    rect(padding+size*2, padding, width/3-padding*2, height-padding*2);
}

void mousePressed() {
    if (redOver) println("Red");
    if (greenOver) println("Green");
    if (blueOver) println("Blue");
}

void update(int x,int y) {
    if (overRed()) {
        redOver=true;
        greenOver=false;
        blueOver=false;
    }
    else if (overGreen()) {
        redOver=false;
        greenOver=true;
        blueOver=false;
    }
    else if (overBlue()) {
        redOver=false;
        greenOver=false;
        blueOver=true;
    }
    else redOver=greenOver=blueOver=false;
}

boolean overRed() {
    int size=width/3;
    if (mouseX>=padding && mouseX<=size-padding && mouseY>=padding &&
mouseY<=size-padding) {
        return true;
    } else {
        return false;
    }
}

boolean overGreen() {
    int size=width/3;
    if (mouseX>=padding+size && mouseX<=size*2-padding && mouseY>=padding
&& mouseY<=size-padding) {
        return true;
    } else {

```



```

        return false;
    }
}
boolean overBlue() {
    int size=width/3;
    if (mouseX>=padding+size*2 && mouseX<=size*3-padding &&
mouseY>=padding && mouseY<=size-padding) {
        return true;
    } else {
        return false;
    }
}
}

```

Kod Tinkercad

```

#define RED 13
#define GREEN 12
#define BLUE 11
#define RED_BUTTON 8
#define GREEN_BUTTON 4
#define BLUE_BUTTON 2

void setup()
{
    pinMode(RED, OUTPUT);
    pinMode(GREEN, OUTPUT);
    pinMode(BLUE, OUTPUT);
    pinMode(RED_BUTTON, INPUT);
    pinMode(GREEN_BUTTON, INPUT);
    pinMode(BLUE_BUTTON, INPUT);
    Serial.begin(9600);
    Serial.println("Enter color (Red | Green | Blue).");
}

void writeRgb(int red, int green, int blue) {
    digitalWrite(RED, red);
    digitalWrite(GREEN, green);
    digitalWrite(BLUE, blue);
}

bool getButton(int buttonPin) {
    return (bool) digitalRead(buttonPin);
}

void loop()
{
    String givenColor="";
    if (Serial.available()) {
        givenColor=Serial.readString();
        givenColor.trim();
        givenColor.toLowerCase();
    }

    bool redOn=getButton(RED_BUTTON);
    bool greenOn=getButton(GREEN_BUTTON);
    bool blueOn=getButton(BLUE_BUTTON);
}

```

```

if (redOn) givenColor="red";
else if (greenOn) givenColor="green";
else if (blueOn) givenColor="blue";

if (givenColor=="red") writeRgb(255,0,0);
else if (givenColor=="green") writeRgb(0,255,0);
else if (givenColor=="blue") writeRgb(0,0,255);
delay(100);
}

```

Приклад

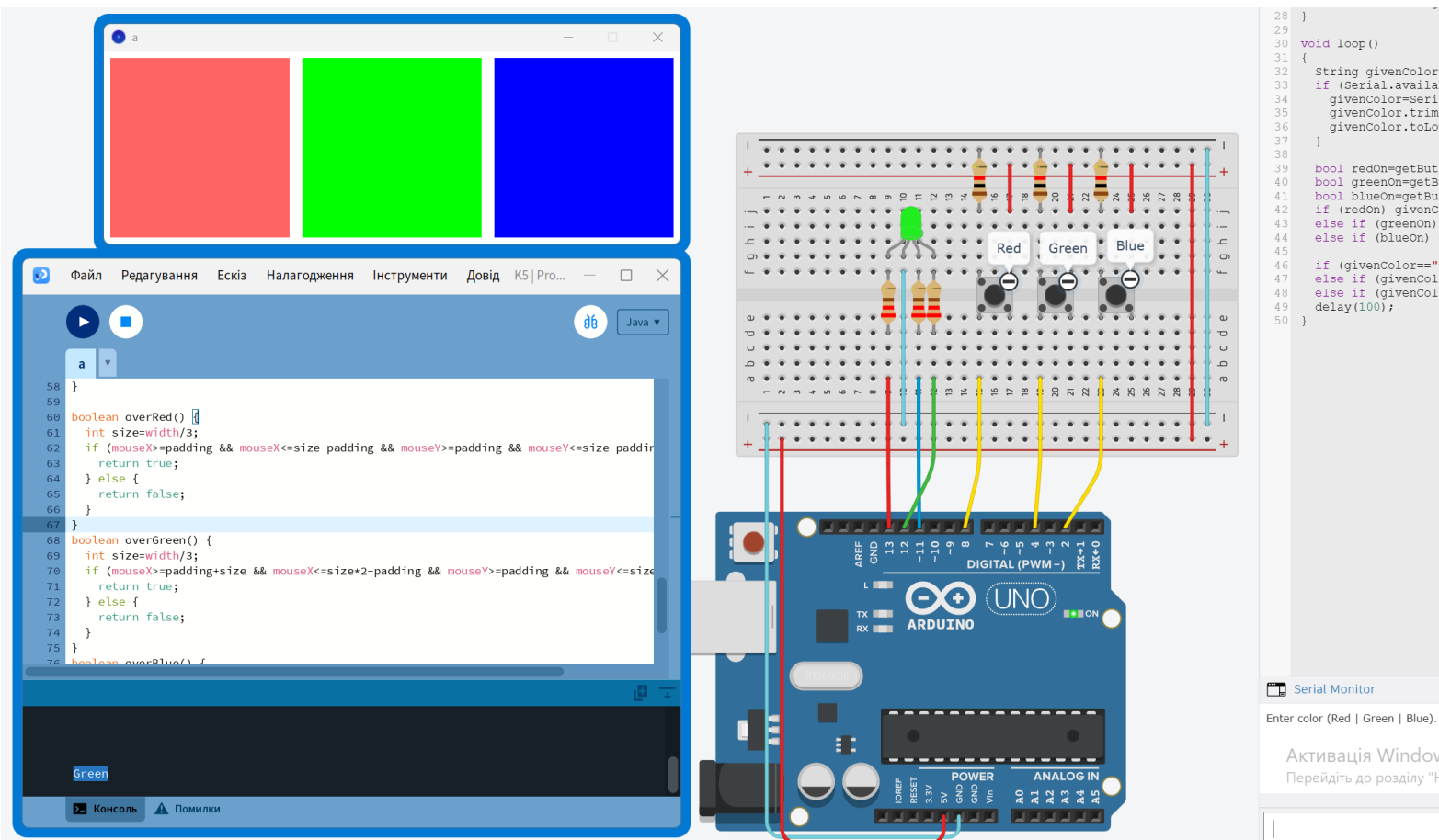


Рисунок 1.1 — Програми в роботі

Відео: https://youtu.be/Io36n1mdw_k.

2 Кут сервоприводу

Код Processing

```

int padding=10;
boolean zeroOver=false;
boolean fortyOver=false;
boolean ninetyOver=false;
color highlightColor = color(245, 245, 245);
int fontSize=50;

void setup() {
  size(300*3, 100*3);
}

void draw() {
  update(mouseX,mouseY);
  background(255);
  stroke(50);
  textSize(fontSize);
  int size=width/3;

  if (zeroOver) fill(highlightColor);
  else fill(255);
  rect(padding, padding, width/3-padding*2, height-padding*2);
  fill(0);
  text("0", (padding+width/3)/2-(fontSize/2), (height)/2);

  if (fourtyOver) fill(highlightColor);
  else fill(255);
  rect(padding+size, padding, width/3-padding*2, height-padding*2);
  fill(0);
  text("45", size*2-size/2-fontSize/2, (height)/2);

  if (ninetyOver) fill(highlightColor);
  else fill(255);
  rect(padding+size*2, padding, width/3-padding*2, height-padding*2);
  fill(0);
  text("90", size*3-size/2-fontSize/2, (height)/2);
}

void mousePressed() {
  if (zeroOver) println("0");
  if (fourtyOver) println("45");
  if (ninetyOver) println("90");
}

void update(int x,int y) {
  if (overZero()) {
    zeroOver=true;
    fortyOver=false;
    ninetyOver=false;
  }
  else if (overFourty()) {
    zeroOver=false;
    fortyOver=true;
    ninetyOver=false;
  }
  else if (overNinety()) {
    zeroOver=false;
    fortyOver=false;
    ninetyOver=true;
  }
}

```

```

    }
    else zeroOver=fourtyOver=ninetyOver=false;
}

boolean overZero() {
    int size=width/3;
    if (mouseX>=padding && mouseX<=size-padding && mouseY>=padding &&
mouseY<=size-padding) {
        return true;
    } else {
        return false;
    }
}

boolean overyFourty() {
    int size=width/3;
    if (mouseX>=padding+size && mouseX<=size*2-padding && mouseY>=padding
&& mouseY<=size-padding) {
        return true;
    } else {
        return false;
    }
}

boolean overNinety() {
    int size=width/3;
    if (mouseX>=padding+size*2 && mouseX<=size*3-padding &&
mouseY>=padding && mouseY<=size-padding) {
        return true;
    } else {
        return false;
    }
}
}

```

Kod Tinkercad

```

#include <Servo.h>
Servo servo;

#define ZERO_BUTTON 7
#define FOURTY_FIVE_BUTTON 6
#define NINETY_BUTTON 5

const int servoPin = 12;

void setup()
{
    Serial.begin(9600);
    servo.attach(servoPin);
    pinMode(ZERO_BUTTON, INPUT);
    pinMode(FOURTY_FIVE_BUTTON, INPUT);
    pinMode(NINETY_BUTTON, INPUT);
}

void loop()
{
    bool zeroClicked=(bool) digitalRead(ZERO_BUTTON);
    bool fourtyFiveClicked=(bool) digitalRead(FOURTY_FIVE_BUTTON);
    bool ninetyClicked=(bool) digitalRead(NINETY_BUTTON);
}

```

```

if (zeroClicked) {
  servo.write(0);
  Serial.println("0");
}
if (fourtyFiveClicked) {
  servo.write(45);
  Serial.println("45");
}
if (ninetyClicked) {
  servo.write(90);
  Serial.println("90");
}
delay(100);
}

```

Приклад

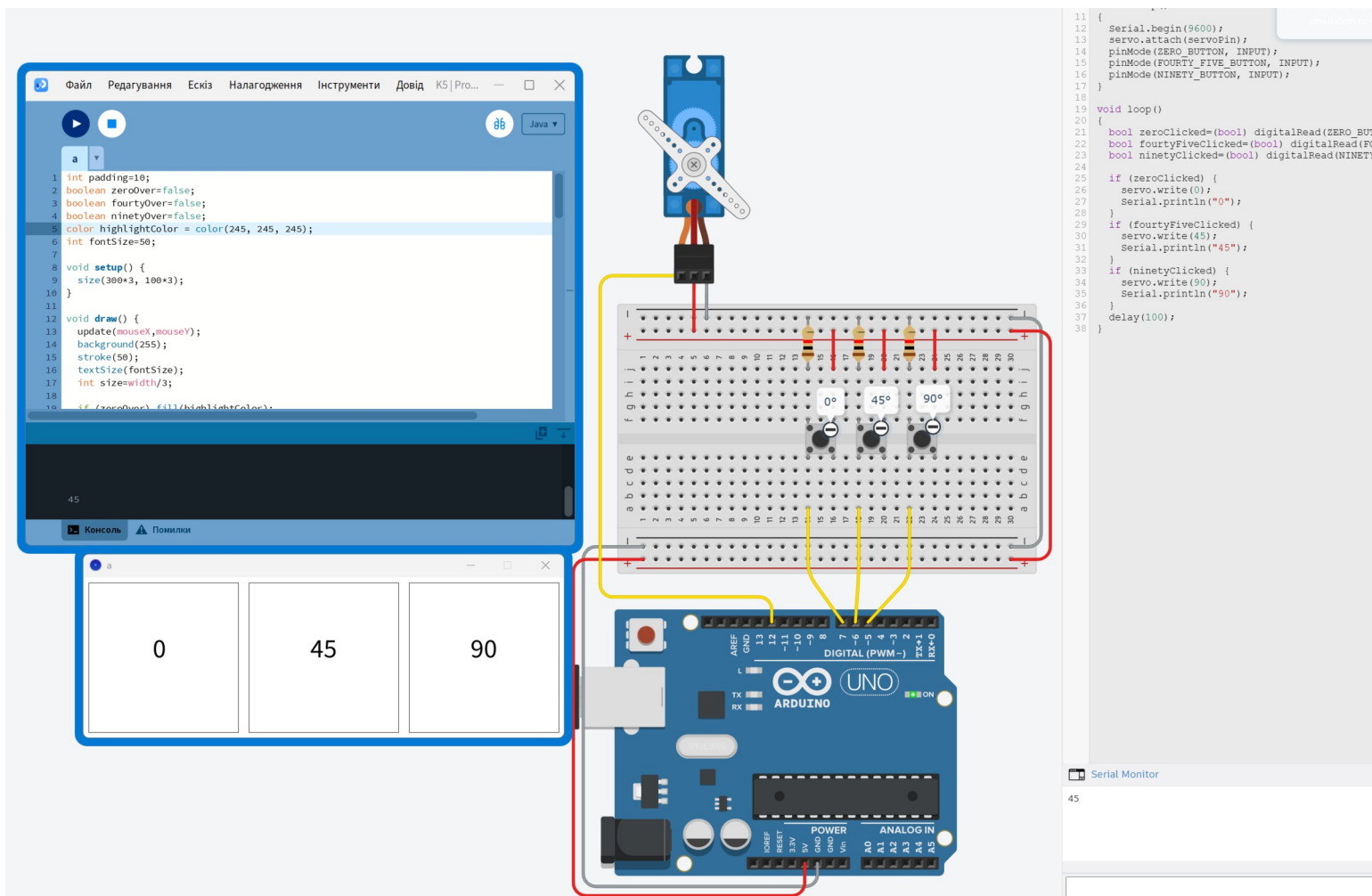


Рисунок 2.1 — Програма в роботі

Відео: <https://youtu.be/R-hZPysj-bE>.

Висновки

По виконанню, благодаттю Господа нашого Ісуса Христа, роботу було навчено створювати у Processing IDE простий графічний інтерфейс який за допомогою послідовного порту «спілкується» з Arduino.

Питання

Чому Arduino та Processing тісно пов'язані?

Processing є продуктом тієї ж компанії.

За допомогою чого передаються дані між Processing та Arduino?

Через порт послідовної взаємодії (Serial Monitor).

Яка бібліотека використовується для роботи з послідовним портом у Arduino та Processing?

Бібліотека Serial, як в четвертій роботі.